

# DOCUMENTACIÓN TÉCNICA PROYECTO

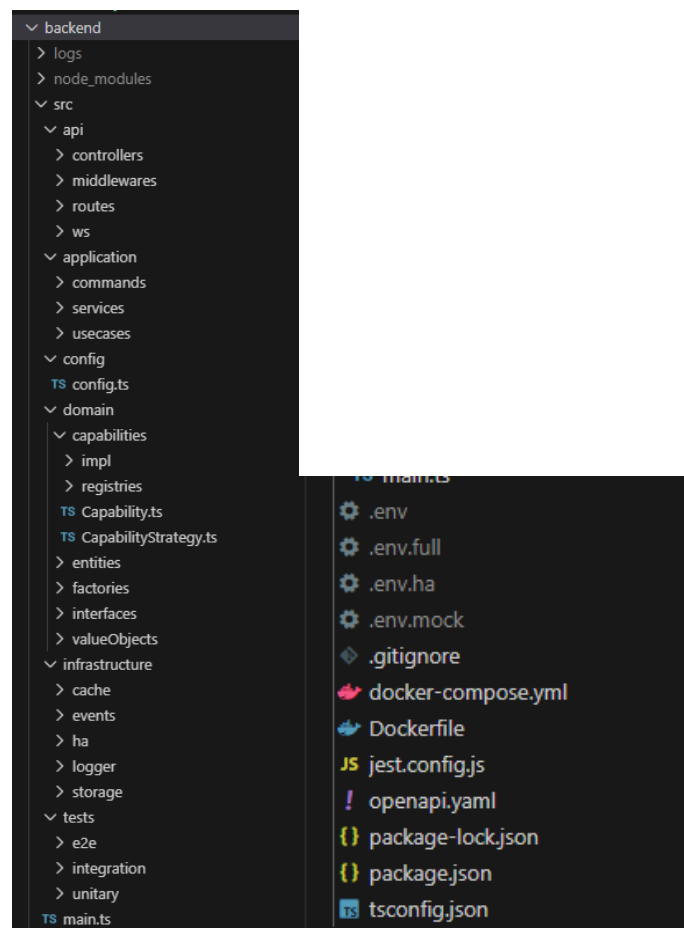
## 1.- Estructura del backend

El backend actúa como **capa intermedia desacoplada** entre:

- Cliente de Unity (VR)
- Hub que gestiona todos los dispositivos de domótica (Home Assistant Green)
- Servicios transversales (cache, eventos, persistencia).

Se ha diseñado con una arquitectura hexagonal, enfocada en la extensibilidad del proyecto (y que se pueda probar fácilmente).

La estructura de la parte backend del proyecto es la siguiente:



## 2.- Capa API

Hace de interfaz de entrada.

### 2.1. Controllers

Está el controlador principal, **DeviceController**, que es el que recibe las peticiones HTTP, comprueban la validez de datos de estas, y delegan en casos de uso.

No tiene lógica de negocio, se puede testear fácilmente, y también se podría cambiar de framework de manera sencilla.

Existe otro controlador, el **HealthController**, que es el que comprueba el estado de la aplicación.

### 2.2. Middlewares

Existe un **authMiddleware**, con una autenticación simple, pensado para un entorno real (de producción), y mejorable también a nivel de seguridad, pero de momento, en la fase de prototipado, pruebas, etc., no se le está dando ningún uso.

También se puede hacer, a futuro, una gestión de roles y permiso, por ejemplo.

### 2.3. Routes

Son los encargados de definir las rutas que conectan con los controllers. Existen dos ficheros:

- **deviceRoutes**
- **healthRoutes**

### 2.4. WS

Solo existe **WebSocketServer**, que es el encargado de enviar estados de dispositivos en tiempo real, y sincronizar con el cliente de Unity (en el futuro).

Está desacoplado del dominio, y solo consume eventos del sistema.

A futuro, se puede añadir, por ejemplo, una autenticación WS.

## 3.- Application

Contiene la lógica de la aplicación.

### 3.1. Commands

Sin implementar todavía, es un inicio de aplicación del patrón command, pensado para acciones que pueden realizar los distintos dispositivos. Cuenta con CommandDispatcher, un comando de prueba, el TurnOnCommand, y un types.

### 3.2. Services

Aquí se incluyen servicios que no pertenecen al dominio puro, y son reutilizables por varios casos de uso.

Este directorio cuenta con:

- **CacheService:** Utiliza Redis para cachear
- **DeviceFilterService:** Sirve para filtrar los dispositivos que aparecen (ya que, por ejemplo, el Home Assistant Green cuenta con muchos sensores virtuales), dejando solo una lista con los dispositivos externos principales.
- **DeviceMonitorService:** Encargado de monitorear los dispositivos añadidos (el estado de cada uno, si se han añadido o quitado, etc.).
- **DeviceRegistry:** Sirve para registrar dispositivos en memoria. Mantiene la consistencia del estado de estos.
- **DeviceService** tiene las acciones principales de devices (obtener la lista, obtener uno concreto, y executeAction, para cuando se implementen)
- **index:** Se exponen las funcionalidades de estos servicios sin acoplar al resto del sistema a la estructura interna.

### 3.3. Use cases

Representan una acción completa del sistema desde el punto de vista del usuario. La aplicación cuenta con tres casos de uso:

- **ListDevicesUseCase**
- **ExecuteDeviceActionUseCase**
- **SyncHADevicesUseCase**

Con el **index** se exponen los casos de uso sin acoplar al resto del sistema a la estructura interna.

## 4.- Domain

Es el núcleo de la aplicación.

### 4.1.- Capabilities

Cada una de las acciones disponibles con las que cuenta un dispositivo. Se pueden extender nuevas acciones sin necesidad de modificar nada de la entidad dispositivo.

Tiene una clase abstracta, **CapabilityStrategy**, que define la base de las Capabilities, y de la que extienden las capabilities concretas (hay algunas creadas, como **OnOfCapability**, pero todavía no están implementadas).

Por otra parte, está el **CapabilityRegistry**, que se encarga de registrar las capabilities de cada dispositivo disponible.

### 4.2.- Entities

Se definen las entidades principales del dominio. La entidad principal es **Device** (clase abstracta), que define a un dispositivo, de la que extienden las demás. De momento, se definieron:

- **GenericDevice**
- **MediaPlayerDevice**
- **SensorDevice**
- **SwitchDevice**

## 4.3.- Factories

El objetivo principal es el de tener una factoría para crear distintos dispositivos en función de las características de estos. Centraliza toda la lógica de creación con el patrón factory.

## 4.4.- Interfaces

Declaran lo que deben hacer los adaptadores, en este caso las interfaces definidas son:

- DeviceStorage: Sirve para la persistencia de dispositivos.
- HAEntity: Define el Home Assistant
- ICache: Sirve para guardar datos en cache
- IDeviceRepository: Define el comportamiento del guardado y obtención de entidades de diversos dispositivos
- IHomeAssistantAPI: Define el comportamiento de la API que gestiona la conexión con el Home Assistant
- ILogger: Define el comportamiento del Logger

## 4.5.- ValueObjects

Representación de:

- DeviceAction**
- DeviceId**
- DeviceState**
- Domain**
- Result**

## 5.- Infrastructure

Cuenta con las implementaciones de las interfaces definidas en la aplicación

### 5.1.- Cache

RedisCache.ts: Adaptador Redis que implementa ICache.

## 5.2.- Events

DeviceEvents.ts: Escucha eventos del dominio y persiste cambios.

events.ts: Event Bus centralizado.

## 5.3.- HA

HAUrlResolver.ts: Normaliza URLs de Home Assistant.

HomeAssistantClient.ts: Adaptador de infraestructura que implementa IHomeAssistantAPI.

## 5.4.- Logger

Index.ts: Expone las funcionalidades del logger

RequestIdMiddleware.ts: Traza peticiones (sirve para debugging).

WinstonLogger.ts: Implementación concreta de ILogger.

## 5.5.- Storage

InMemoryDeviceRepository.ts: Repositorio en memoria. Sirve para tests, mocking, y no tiene dependencias externas.

InMemoryDeviceStorage.ts: Similar al anterior, pero orientado a la memoria en tiempo de ejecución.

RepoMock.ts: Mock de dispositivos para pruebas sin el HA.

## 6.- Archivo main.ts

Se conectan todas las capas. Está organizado de la siguiente manera:

1.- Se definen las rutas en función del entorno en el que se esté ejecutando.

```
export const startServer = async () => {  
  /* ----- ROUTES ----- */  
  
  if (config.env === "production") {  
    app.use("/devices", authMiddleware, deviceRoutes);  
  } else {  
    app.use("/devices", deviceRoutes);  
  }  
}
```

2.- Intenta conectarse a Redis, si está disponible

```
// ----- CACHE / REDIS -----  
  
if (config.redisUrl && config.redisUrl.trim() !== "") {  
  const r = new RedisCache(config.redisUrl);  
  try {  
    await r.connect();  
    cacheService.setCacheAdapter(r);  
  } catch (err) {  
    logger.warn("Redis not available, continuing without cache")  
  }  
} else {  
  logger.info("Redis disabled (no REDIS_URL provided)");  
}
```

3.- Inicializa el DeviceRegistry

```
// ----- REGISTRY -----  
  
await DeviceRegistry.init();
```

4.- Comprueba si hay que sacar los datos mockeados, o se tiene que conectar al HA. En este último caso, se inicializa el DeviceMonitorService, para estar al corriente de si los dispositivos han sufrido algún cambio.

```
// ----- DATA SOURCE SELECTION -----

if (config.useMockData) {
  logger.info("Starting backend in MOCK mode");

  const devices = await repoMock.getAll();
  devices.forEach((d) => DeviceRegistry.addDevice(d));
} else {
  logger.info("Starting backend connected to Home Assistant");

  const haClient = new HomeAssistantClient(
    config.haUrl,
    config.haToken
  );

  await haClient.connect();
  DeviceRegistry.setHA(haClient);

  const deviceRepository = new InMemoryDeviceRepository();
  You, yesterday • feat(optimized the docker-compose and Upd.
  const sync = new SynchADevicesUseCase(
    haClient,
    deviceRepository
  );

  await sync.execute();

  const monitor = new DeviceMonitorService(
    DeviceRegistry,
    haClient
  );

  monitor.start();
}
```

5.- Inicializa el websocketServer y el servidor

```
// ----- HTTP + WS -----

server = http.createServer(app);

if (config.enableWs) {
  initWebSocketServer(server);
}

server.listen(config.port, () => {
  logger.info(`Server running on port ${config.port}`);
});

return server;
};
```



## 7.- Testing

De momento, el proyecto cuenta con los siguientes tests:

- E2e (backend\_http.test.ts, y backend\_ws.test.ts, para comprobar la conexión con el servidor y el websocket).
- Integración (DeviceRegistry.integration.test.ts, que comprueba que se registren dispositivos y reciba correctamente la lista de los mismos).
- Unitarios (son los más extensos, cuenta con 14 ficheros de tests que prueban diversas clases a nivel individual, falta revisar si se podrían añadir o mejorar algunos)

## 8.- Configuración

Existen 4 ficheros de configuración, aunque de momento en los que tenemos que fijarnos son en .env.ha y en .env.mock.

.env.ha sirve para levantar el entorno con un HA disponible, y .env.mock mockea el HA. Ambos tienen REDIS por defecto.

Los entornos, mediante la configuración definida en el docker-compose.yml, están preparados para, según el comando que se ponga, se inicializa un entorno u otro.

## 9.- Docker

Se ha configurado un Dockerfile y un docker-compose.yml para levantar el proyecto de manera más sencilla.

El Dockerfile solo prepara el entorno, es muy simple, sin embargo, el docker-compose.yml tiene un par de funcionalidades interesantes (se han documentado también en el README del proyecto). Si se ejecuta con el profile en dev-ha (copiando antes la configuración del fichero .env.ha al fichero .env), se levanta en modo conexión con un HA. Si se ejecuta con el dev-mock (haciendo un cp de .env.mock a .env), entonces se levanta el entorno en modo mockeado.

## 10.- Ejecución

```
### Docker

- Start environment with real HA
1. `cp .env.ha .env`
2. `docker-compose --profile dev-ha up --build`

- Start environment with a mock HA
1. `cp .env.mock .env`
2. `docker-compose --profile dev-mock up --build`
```

Si se levanta el entorno mockeado, se verá un mensaje de confirmación de que el server está levantado.

Si se levanta el entorno del HA, además de lo anterior, cada 60 segundos se enviará un mensaje que mostrará los dispositivos conectados.

```
PS D:\workspace\Git\Mixed-Reality-Smart-Home-Control-System\backend> docker-compose --profile dev-ha up --build
backend-1 | {"level":"info","message":"Starting backend connected to Home Assistant","timestamp":"2025-12-15T19:03:39.922Z"}
backend-1 | {"level":"info","message":"Connecting to HA","timestamp":"2025-12-15T19:03:39.922Z","url":"ws://homeassistant.local:8123/api/websocket"}
backend-1 | {"level":"info","message":"WS open","timestamp":"2025-12-15T19:03:40.032Z"}
backend-1 | {"level":"info","message":"WS Server initialized at /ws","timestamp":"2025-12-15T19:03:40.048Z"}
backend-1 | {"level":"info","message":"Server running on port 3000","timestamp":"2025-12-15T19:03:40.049Z"}
backend-1 | {"level":"info","message":"[INFO] CHECKING DEVICES","timestamp":"2025-12-15T19:04:40.050Z"}
backend-1 | {"id":"media_player.dormitorio","level":"info","message":"Device updated","timestamp":"2025-12-15T19:04:40.059Z"}
backend-1 | {"id":"media_player.ta_chiquita","level":"info","message":"Device updated","timestamp":"2025-12-15T19:04:40.059Z"}
backend-1 | {"id":"switch enchufe_tapo","level":"info","message":"Device updated","timestamp":"2025-12-15T19:04:40.059Z"}
█
```